Journal of Machine Learning Research 1 (2026) 1–5 Submitted 7/26; Published

kodama-cpp: Cross-Validated Accuracy Maximization on CPU, CUDA, and Apple Metal

Moussa Kassim[1,2] MOUSSA.KASSIM@ICGEB.ORG
Martin Ocharo[1,2] MARTIN.OCHARO@ICGEB.ORG
Dalia Ahmed[1] DALIA.AHMED@ICGEB.ORG
Dupe Ojo[1] DUPE.OJO@ICGEB.ORG
Alessia Vignoli[3,4] VIGNOLI@CERM.UNIFI.IT
Leonardo Tenori[3,4] TENORI@CERM.UNIFI.IT
Stefano Cacciatore[1,2] STEFANO.CACCIATORE@ICGEB.ORG
[1] *Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology (ICGEB), Cape Town 7925, South Africa*
[2] *Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*
[3] *Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy*
[4] *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*

Editor: To be assigned

Abstract

KODAMA searches for latent structure by maximizing the held-out predictability of evolving labels. We present `kodama-cpp`, a standalone C++17 implementation that preserves this objective across multicore CPU, NVIDIA CUDA, and Apple Metal. One float32 core owns folds, labels, classifier workspaces, and graph outputs. It exposes KNN and SIMPLS followed by latent-space LDA; backend implementations provide neighbor search, k-means, label-aware cross-products, reusable workspaces, and strict backend identity without runtime dependence on R, Python, FAISS, cuVS, RAFT, or Armadillo. Independent runs may sample different landmarks, and each proposal receives exactly one cross-validation evaluation. The matrix/graph API also provides PCA and UMAP/openTSNE, with thin R and Python wrappers. Validation separates classifier parity, kernel and complete runtimes, and external-label diagnostics.

Keywords: KODAMA, cross-validation, unsupervised learning, heterogeneous computing, manifold learning

1 Introduction

KODAMA treats a sample-label vector as an optimization variable rather than observed truth. A candidate labeling is useful when a classifier trained on some samples can reproduce it on held-out samples. The original method and R package established this cross-validated accuracy principle and used an ensemble of optimized label vectors to correct pairwise dissimilarities before visualization (Cacciatore et al., 2014, 2017). The method is therefore related to prediction-strength and stability validation, but differs by placing held-out predictability inside the label search itself (Tibshirani and Walther, 2005).

Repeated fitting also makes KODAMA a systems problem: graphs, folds, class encodings, and projections recur across proposals. `kodama-cpp` moves this work into one float32 core with a common state machine and R/Python contract. It is a portable implementation of established mathematics, not a replacement objective.

2 KODAMA objective and search

For data $X \in \mathbb{R}^{n\times p}$, labels y, folds Π, and classifier family F, let

$$A(y; X, F, \Pi) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[y_i = \widehat{y}_i^{(-\Pi(i))}\right]. \tag{1}$$

KODAMA searches for label vectors with high A; it does not interpret A as external accuracy. The current implementation uses either KNN or SIMPLS plus LDA in the requested feasible latent dimension (de Jong, 1993). Fold assignments remain fixed within a run, and constrained samples move as one group.

Run r uses seed $s + r$, up to L landmarks, and `splitting` k-means labels. If $L \geq n$, the historical rule $\lceil 0.75n \rceil$ is retained; initial classes default to 100 for $n < 40000$ and 300 otherwise. Held-out predictions propose grouped relabelings whose maximum size decreases smoothly,

$$q_{\max}(t) = 1 + \left\lfloor (G-1)\left[1 - p_t^2(3-2p_t)\right]\right\rfloor, \qquad p_t = \frac{t+1}{T+1}, \tag{2}$$

with q sampled uniformly from $1, \ldots, q_{\max}(t)$. Transition proposals absorb source labels preferentially predicted as a target without permitting one class; PLS–LDA also coarsens unstable fragments.

The proposed vector receives exactly one new CV pass. Raw A is reported, while the default acceptance score guards against trivial predictability:

$$S(y) = A(y)\sqrt{1 - \sum_k p_k^2} - (1 - A(y))P(y), \tag{3}$$

where p_k is a class proportion and $P = 0$ for KNN; for PLS–LDA, P is the larger of normalized label entropy and an effective-class fragmentation term. A proposal improving the best S is retained. The current chain can also accept a worse proposal with probability $\exp[(S_{new} - S_{cur})/\tau_t]$, where $\tau_t = 0.10(1 - A_{cur})(1 - t/T)$. These grouped, guarded search dynamics extend the historical implementation but use only CV predictions, never reference labels.

After T cycles, labels are projected from landmarks to all samples. Across runs, an original graph edge (i, j) has agreement a_{ij} equal to the fraction of valid runs assigning its endpoints the same label. Its corrected distance is $d'_{ij} = (1 + d_{ij})/a_{ij}^2$; zero agreement removes the edge. This sparse graph is the KODAMA representation supplied to visualization.

> **One independent run.** Initialize landmarks, labels, folds, and one CV prediction. For $t = 1, \ldots, T$, propose grouped/transition moves, evaluate once by CV, update the best by S, and update the current state by cooling. Return the best raw accuracy and labels. Repeat for M independent runs, project labels, and reweight the shared graph by agreement.

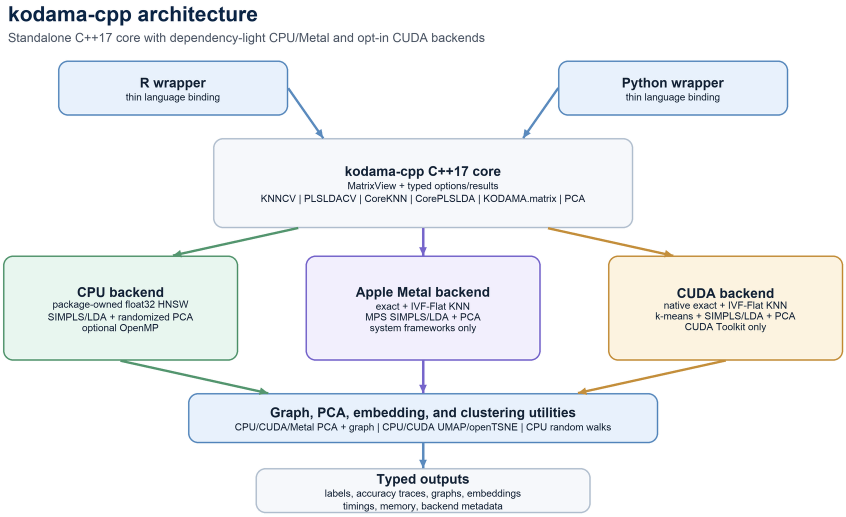

Figure 1: The C++17 core owns KODAMA state and exposes one typed API to R and Python. Backend-specific kernels share the same folds, proposals, component-count semantics, and result contract.

3 Standalone heterogeneous implementation

Figure 1 shows the ownership boundary. CPU provides package-owned HNSW (Malkov and Yashunin, 2020), SIMPLS/LDA, PCA, graph operations, and UMAP/openTSNE. CUDA provides exact/IVF KNN, batched k-means, label-aware SIMPLS/LDA, PCA, and embedding kernels; Metal provides exact/IVF KNN, k-means, MPS-assisted SIMPLS/LDA, and PCA. Unavailable accelerators raise errors rather than silently executing CPU code.

PLS–LDA avoids dense one-hot responses by computing $X^\top Y$ from class sums and compacting active labels each cycle. Fold indices and workspaces are reused. The requested feasible component count is evaluated without internal selection. Results record predictions, folds, confusion matrices, label ensembles, resources, search parameters, and the executing backend.

For latent training scores $T \in \mathbb{R}^{n\times q}$, class counts n_c, means μ_c, and C active classes, the pooled within-class covariance is

$$\Sigma = \frac{T^\top T - \sum_{c=1}^{C} n_c \mu_c \mu_c^\top}{\max(1, n - C)}. \tag{4}$$

The implementation solves $\Sigma w_c = \mu_c$ by Cholesky factorization and triangular substitution, without explicitly inverting Σ. It adds $\lambda = \rho s$ to the diagonal, where $s = \mathrm{tr}(\Sigma)/q$ when finite and positive (otherwise $s = 1$), and tries $\rho \in \{10^{-8}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$ in order, advancing only after factorization failure. Prediction uses $\delta_c(t) = t^\top w_c - \frac{1}{2}\mu_c^\top w_c + \log(n_c/n)$. Thus the ridge sequence is a deterministic numerical safeguard rather than a tuned model parameter.

Table 1: Selected implementation validation. Accuracy is CPU/accelerator; runtime is seconds.

Platform	Dataset/kernel	CPU	Accel.	Speedup	Accuracy
CUDA	MNIST KNNCV	76.769	4.753	16.2	.974/.973
CUDA	MetRef KODAMA, $M = T = 100$	340.451	177.131	1.92	.992/.992
CUDA	MetRef PLSLDACV	1.294	0.308	4.20	.992/.993
Metal	MetRef KNNCV	11.145	0.026	425.0	.827/.827
Metal	MetRef PLSLDACV	0.790	0.202	3.90	.991/.990

The alternative graph API lets KNN consume supplied neighbors; graph-only PLS–LDA uses a documented self-tuning normalized Laplacian. PCA is auxiliary. Pinned fastEmbedR UMAP/openTSNE kernels use matched classic/corrected contracts and direct float32 CSR graphs (McInnes et al., 2018; van der Maaten and Hinton, 2008).

4 Validation and scope

CTest checks numerical, float32, backend, graph, and frozen `0.1.0` API contracts; wrappers test matrix, graph, PCA, and visualization calls. GitHub Actions passed CPU jobs on Linux and macOS, native Metal, R/Python, coverage, and documentation. LLVM CPU coverage was 63.58% by line and 57.03% by branch; accelerators retain hardware-specific tests. Table 1 reports within-platform measurements.

Five-run MetRef PLSLDACV medians gave 3.90x Metal and 4.20x CUDA speedups over their four-thread hosts; accuracy differed by at most one of 873 samples. Complete $M = T = 100$ KODAMA gave a 1.92x CUDA speedup. Rank-one matrix–vector operations and omission of overwritten random initialization reduce CUDA work without changing SIMPLS or LDA equations.

Matched MetRef KNN ($M = T = 100$) took 610.5 seconds in KODAMA R 2.4.1 and 965.3/235.5/2.36 seconds in kodama-cpp CPU1/CPU4/CUDA. A non-parity $M = T = 20$ current-R/kodama-cpp PLS–LDA CPU4 check took 344.914/822.989 seconds (ARI .7004/.6430); CUDA was omitted at graph $k = 500 > 256$.

Raising T from 20 to 100 improved median accuracy and fragmentation for USPS and MetRef PLS–LDA, while MetRef KNN stayed over-coarsened. At $M = 100$, worst-case agreement-edge standard error is .05 and every $M = 50$ graph correlated at least .9888 with $M = 100$. ARI was nonmonotone; a five-dataset panel retained both positive and negative silhouette changes.

5 Availability and limitations

Version `0.1.0` is MIT licensed at `https://github.com/tkcaccia/kodama-cpp`; Metal retains an Apache-2.0 exception. Provenance, API tests, and documentation are archived; the final DOI is pending. Repeated CV dominates and M runs are not fully device resident. Approximate ties may vary; CUDA graph $k \leq 256$. Graph-only PLS–LDA is a spectral surrogate.

References

Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.

Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.

Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.